

GRAPHS – FOUNDATIONS, TYPES & REPRESENTATIONS

- ★ **Tree $\subset$ Graph** (Trees are special graphs!)
- ★ **Grid = Graph** (Cells are nodes, neighbors are edges)
- ★ **Most interview problems hide graphs** → Think Graph first!
- ★ **Adjacency List** is used 95% of the time

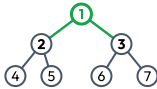
1. PLAIN ENGLISH — WHAT IS A GRAPH?

- A Graph is a collection of nodes connected by edges
- Trees are special graphs (no cycles, connected, has a root)
- Grids are graphs (cells as nodes, neighbors as edges)
- Most interview problems hide graphs (think graph first!)

A Tree $\subset$ Graph. A Grid is a Graph. Most problems are secretly graphs.

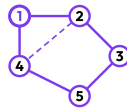
2. VISUAL EXAMPLES — TREE VS GRAPH VS GRID

TREE (Special Case of Graph)



No cycles, Connected, Has root

GENERAL GRAPH



May have cycles, May be disconnected

GRID (2D Graph)

(0,0)	(0,1)	(0,2)
(1,0)	(1,1)	(1,2)

Cells = Nodes, Neighbors = Edges

3. BRAIN SWITCH — WHAT HIDES A GRAPH?

- Cities & Roads → **Graph**
- Computer Network → **Graph**
- Friends/Social Network → **Graph**
- Maze/Game Map → **Grid Graph**
- Flights/Airlines → **Graph**
- Map → **Graph**
- Courses/Prerequisites → **Graph (DAG)**
- Dependencies/Tasks → **DAG**
- Files/Folder Structure → **Tree (Graph)**

Whenever you see **RELATIONSHIPS** → Think **GRAPH**

4. GRAPH TYPES (KNOW ALL 8!)

UNDIRECTED

Edges have no direction. Ex: Friends, Roads

DIRECTED

Edges have direction. Ex: Flights, Instagram follows

WEIGHTED

Edges have weights/costs. Ex: Roads with distances

UNWEIGHTED

All edges equal weight. Ex: Social Network

CONNECTED

Path between every pair of nodes. Ex: Network

DISCONNECTED

Not all nodes reachable. Ex: Components

CYCLIC

Contains at least one cycle. Ex: Dependencies

DAG (Directed Acyclic)

Directed + no cycles. Ex: Course Schedule, Tasks

5. GRAPH TERMINOLOGY

Vertex (Node)	Basic unit of graph (A, B, C...)
Edge	Connection between two nodes
Degree(A)	Number of edges connected to A
Neighbor	Any connected adjacent node
Weight	Cost of traversing an edge
Path	Sequence of nodes via edges (A→C→E)
Cycle	Path that starts and ends at same node
Component	Set of nodes that are reachable from each other
In-degree	Count of incoming edges to a node (directed)
Out-degree	Count of outgoing edges from a node

7. GRAPH REPRESENTATIONS (OVERVIEW)

Representation	Typical Input	Memory	Notes
Adjacency List	[[1,2],[2,3]...]	$O(V+E)$	★★★★★ Most Common (95%)
Adjacency Matrix	$n \times n$ 2D matrix	$O(V^2)$	Dense graphs, $O(1)$ edge check
Edge List	[u,v,w] pairs	$O(E)$	Kruskal's, some problems
Grid	char[][] / int[][]	$O(m \times n)$	Implicit graph (dirs[])
Node Graph	Nodes with Node[]	$O(V+E)$	Clone Graph, complex structures

★ **Adjacency List is the most commonly used representation. Build it first!**

Build Adjacency List (Java):

```
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
graph.get(u).add(v); // directed
graph.get(v).add(u); // also for undirected
```

6. TREE VS GRAPH (QUICK TABLE)

Aspect	Tree	General Graph
Root	Has a root	No root (usually)
Parent	Each node has exactly one parent	A node can have multiple parents
Cycles	No cycles	May have cycles
Connected	Always connected	May be disconnected
Edges	$n-1$ edges	Any number
Need visited[]	Usually NOT needed	Usually needed

A Tree is always a Graph. A Graph is NOT always a Tree.

11. INTERVIEW THINKING (BEFORE WRITING CODE)

- 1 What is the input? Understand the given data.
- 2 Is it Directed? Do edges have direction?
- 3 Is it Weighted? Do edges have weights/costs?
- 4 Is it Connected? Can all nodes reach each other?
- 5 Which Representation should I build? Adj List, Matrix, Grid, etc.
- 6 Which Algorithm family will probably solve it? DFS / BFS / Topo Sort / Union

9. PATTERN RECOGNITION (INTERVIEW PROBLEM → EXPECTED GRAPH TYPE)

Interview Says	Expected Type
Cities & Roads	Undirected, Weighted
Flights / Airlines	Directed, Weighted
Courses/Prerequisites	Directed DAG
Grid / Matrix	Grid Graph (4/8 dir)

Interview Says	Expected Type
Network / Servers	Undirected or Directed
Friends / Social	Undirected, Unweighted
Maze / Game Map	Grid Graph
Dep. / Build Order	Directed DAG

10. Common Mistakes

- ✗ Forgetting graphs may be disconnected
- ✗ Confusing Tree with Graph
- ✗ Ignoring direction (Directed vs Undirected)
- ✗ Ignoring weights
- ✗ Choosing wrong representation
- ✗ Not defining neighbors clearly

8. AHA Moments

- ★ A Tree is a Graph
- ★ A Grid is a Graph
- ★ Representation changes, algorithm usually doesn't
- ★ Outer loop = Multiple Components
- ★ Most problems hide the graph — identify it first!

★ PROBLEM → INPUT → REPRESENTATION → HELPER STRUCTURE → ALGORITHM → TINY LOGIC CHANGE → DRY RUN → OPTIMIZE

Next: DFS Templates + BFS Templates →

GRAPHS – MASTER DFS TEMPLATES & PROBLEM GUIDE

Graph DFS · Grid DFS · Node DFS · Outer Loop DFS · Border DFS · 7 DFS Problems Solved

G=Graph DFS (AdjList) Gd=Grid DFS N=Node Graph DFS OL=Outer Loop DFS Bd=Border DFS ★ Only the CHANGED LINES differ between templates!

GRAPH DFS (ADJACENCY LIST) ★★★★★

Input:

Adjacency List

Caller:

dfs(start)

Visited:

boolean[]

Return:

void/boolean/int

Used In:

- Valid Path
- Connected Components
- Graph Valid Tree

```
void dfs(int node, List<List<Integer>>> graph, boolean[] visited) {
    visited[node] = true; // mark
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) {
            dfs(neighbor, graph, visited);
        }
    }
}
// AHA: Neighbors come from graph.get(node)
```

GRID DFS ★★★★★

Input:

Grid (m×n)

Caller:

dfs(r,c)

Visited:

boolean[][]

Helpers:

int[][] dirs

Used In:

- Number of Islands
- Flood Fill
- Max Area of Island
- Surrounded Regions

```
void dfs(int r, int c, char[][] grid, boolean[][] visited) {
    int m = grid.length, n = grid[0].length;
    // 1. Boundary check
    if (r < 0 || r ≥ m || c < 0 || c ≥ n) return;
    // 2. Visited or blocked cell
    if (visited[r][c] || grid[r][c] == '0') return;
    visited[r][c] = true; // mark
    // 4. Explore 4 directions
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    for (int[] d : dirs) {
        dfs(r + d[0], c + d[1], grid, visited);
    }
}
// AHA: Grid is an implicit graph!
```

WHAT ACTUALLY CHANGES? (30-SECOND REVISION)

Template	Special Change
Graph DFS	Neighbors from graph.get(node)
Grid DFS	Movement using dirs[] + boundary check
Node DFS	Visited is HashMap<Node,Node>
Outer Loop DFS	Starts DFS from every unvisited node
Border DFS	Starts DFS from borders / multiple sources

The DFS never changes. Only WHERE it starts changes.

RETURN TYPE GUIDE

Type	Purpose	Problems
void	Mark/Traverse	Flood Fill, Surrounded Regions
boolean	Search/Decision	Valid Path, Detect Cycle
int	Count/Aggregate	Islands, Max Area of Island
Node	Build new structure	Clone Graph
List	Collect results	All Paths, Topo Sort

★ Changing the return type usually changes the solution!

OUTER LOOP DFS (DISCONNECTED GRAPHS)

Used In:

Connected Components, Number of Provinces, Number of Islands

```
// For Graph (Adjacency List):
int count = 0;
boolean[] visited = new boolean[n];
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i, graph, visited); // use Graph DFS count++;
    }
}
return count; // number of components
// AHA: Outer loop discovers disconnected graphs!
```

BORDER DFS (MULTI-START) ★★★★★

Used In:

Pacific Atlantic, Surrounded Regions. Run DFS from border cells/nodes.

```
// Top & Bottom rows
for (int c = 0; c < n; c++) {
    if (!visited[0][c]) dfs(0, c, grid, visited);
    if (!visited[m-1][c]) dfs(m-1, c, grid, visited);
}
// Left & Right columns
for (int r = 0; r < m; r++) {
    if (!visited[r][0]) dfs(r, 0, grid, visited);
    if (!visited[r][n-1]) dfs(r, n-1, grid, visited);
}
// AHA: Multiple DFS starting points from borders!
```

NODE GRAPH DFS (CLONE GRAPH)

Input:

Node with neighbors[]

Visited:

HashMap<Node,Node>

```
Node dfs(Node node, HashMap<Node,Node> map) {
    if (node == null) return null;
    if (map.containsKey(node)) return map.get(node);
    Node clone = new Node(node.val);
    map.put(node, clone);
    for (Node nei : node.neighbors)
        clone.neighbors.add(dfs(nei, map));
    return clone;
}
// AHA: HashMap replaces visited[]
```

Graph DFS Problem Guide — Same DFS. Different Problems. Only the Changing Parts.

P1 VALID PATH (LC 1971)

Base: Graph DFS

CHANGED LINES

```
// 1. Return true if destination reached
if (node == destination) return true;
// 2. Change return type to boolean
if (dfs(neighbor, destination, graph, visited))
    return true;
```

AHA: Stop early when destination found.

Time: O(V+E) Space: O(V)

P2 CONNECTED COMPONENTS

Base: Outer Loop DFS

CHANGED LINES

```
// 1. Outer loop to cover all nodes
int count = 0;
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        count++;
        dfs(i, graph, visited);
    }
}
// 2. DFS remains same (void)
```

AHA: Each unvisited node starts a new component.

Time: O(V+E) Space: O(V)

P3 NUMBER OF ISLANDS (LC 200)

Base: Outer Loop Grid DFS

CHANGED LINES

```
// 1. Outer loop over entire grid
for (int r = 0; r < m; r++) {
    for (int c = 0; c < n; c++) {
        if (grid[r][c] == '1' && !visited[r][c]) {
            count++;
            dfs(r, c, grid, visited);
        }
    }
}
// 2. DFS marks all connected land
```

AHA: Every unvisited land cell starts a new island.

Time: O(m×n) Space: O(m×n)

P4 FLOOD FILL (LC 733)

Grid DFS

CHANGED LINES

```
// 1. Store original color
char origColor = image[sr][sc];
// 2. Color check
if (image[r][c] != origColor) return;
// 3. Change the color
image[r][c] = newColor;
// 4. Rest same (visited=4 directions)
```

AHA: Only change pixels with the original color.

Time: O(m×n) Space: O(m×n)

P5 PACIFIC ATLANTIC (LC 417)

Border DFS

CHANGED LINES

```
// 1. Run DFS from all Pacific borders
for (int r=0; r<m; r++) dfs(r, 0, heights, pacific);
for (int c=0; c<n; c++) dfs(0, c, heights, pacific);
// 2. Run DFS from all Atlantic borders
for (int r=0; r<m; r++) dfs(r, n-1, heights, atlantic);
for (int c=0; c<n; c++) dfs(m-1, c, heights, atlantic);
// 3. Collect intersection
if (pacific[r][c] && atlantic[r][c]) add to answer;
```

AHA: Flow is reverse — start DFS from oceans.

Time: O(m×n) Space: O(m×n)

P6 SURROUNDED REGIONS (LC 130)

Border DFS

CHANGED LINES

```
// 1. Mark boundary connected '0's
// Run DFS from all borders; mark as 'safe'
// 2. Flip remaining '0's
for (int r=0; r<m; r++)
    for (int c=0; c<n; c++)
        if (board[r][c]=='0' && !safe[r][c])
            board[r][c] = 'X';
```

AHA: Only border-connected regions are safe. Two-step thinking!

Time: O(m×n) Space: O(m×n)

P7 CLONE GRAPH (LC 133)

Node DFS

CHANGED LINES

```
// If node not in map → create clone
// For each neighbor → recurse and add edge
// Already handled in Node DFS template
// KEY: Use HashMap for visited (avoids infinite loop)
// If node in map → return clone (already visited)
```

AHA: HashMap avoids infinite loops in cycles.

Time: O(V+E) Space: O(V)

★ MASTER IDEA: DFS logic stays the same. Only the caller, input, return type and tiny logic change!

Next: BFS Templates + BFS Problems →

GRAPHS – MASTER BFS TEMPLATES & PROBLEM GUIDE

Graph BFS · Grid BFS · Multi-Source BFS · Level Order BFS · Distance BFS · 6 Problems Solved

Gb=Graph BFS (AdjList) Gr=Grid BFS Ms=Multi-Source BFS Lv=Level Order BFS Ds=Distance BFS ★ Queue + Visited[] are the heart of BFS!

GRAPH BFS (ADJACENCY LIST) ★★★★★

Input: Adjacency List
Caller: bfs(start)
Visited: boolean[]
Queue: Queue<Integer>
Used In: Valid Path, Connected Components, Shortest Path (Unweighted)

```
void bfs(int start, List<List<Integer>> graph, boolean[] visited) {
    Queue<Integer> q = new LinkedList<>();
    q.offer(start);
    visited[start] = true;
    while (!q.isEmpty()) {
        int node = q.poll();
        // Process node here
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.offer(neighbor);
            }
        }
    }
}
// AHA: Explores level by level outwards.
```

GRID BFS ★★★★★

Input: Grid
Caller: bfs(r, c)
Visited: boolean[][]
Queue: Queue<int[]>
Used In: Shortest Path in Binary Matrix, Rotting Oranges, Walls & Gates

```
void bfs(int r, int c, int[][] grid, boolean[][] visited) {
    int m = grid.length, n = grid[0].length;
    Queue<int[]> q = new LinkedList<>();
    q.offer(new int[]{r, c});
    visited[r][c] = true;
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    while (!q.isEmpty()) {
        int[] curr = q.poll();
        for (int[] d : dirs) {
            int nr = curr[0] + d[0];
            int nc = curr[1] + d[1];
            if (nr >= 0 && nr < m && nc >= 0 && nc < n && !visited[nr][nc]) {
                visited[nr][nc] = true;
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
// AHA: Move using dirs[].
```

WHAT ACTUALLY CHANGES? (30-SECOND REVISION)

Template	Special Change
Graph BFS	Neighbors from graph.get(node)
Grid BFS	Movement using dirs[] + bounds check
Multi-Source	Initial queue contains ALL sources
Level Order	Use queue size to process by level
Distance BFS	Track level/distance for each node

★ First time you visit a node in BFS, it is ALWAYS the shortest path!

LEVEL ORDER BFS ★★★★★

Idea: Process one level at a time using queue size.
Key: int size = q.size();

```
while (!q.isEmpty()) {
    int size = q.size(); // nodes in current level
    for (int i = 0; i < size; i++) {
        int node = q.poll();
        // process node at this level
        for (int nei : graph.get(node)) {
            if (!visited[nei]) {
                visited[nei] = true;
                q.offer(nei);
            }
        }
    }
}
// AHA: Queue size gives number of nodes at current level.
```

MULTI-SOURCE BFS ★★★★★

Idea: Push ALL sources first, then BFS. Distances spread outward simultaneously.

```
Queue<int[]> q = new LinkedList<>();
// 1. Add all sources
for (int[] src : sources) {
    q.offer(src);
    visited[src[0]][src[1]] = true;
}
// 2. Standard BFS (often with Level Order)
// AHA: Multiple starting points become one BFS!
```

RETURN TYPE GUIDE

Type	Purpose	Problems
void	Mark/Traverse	Flood Fill, Walls & Gates
boolean	Search/Check	Valid Path, Bipartite
int	Dist/Steps	Shortest Path, Min Steps
int[][]	Store Dists	01 Matrix, Nearest Exit

COMMON MISTAKES

- ✗ Forgot to mark visited! (Infinite loop)
- ✗ Marked visited *after* taking from queue (Too slow! Mark when adding).
- ✗ Forgot queue size when doing level order.
- ✗ Used DFS instead of BFS for shortest path.

Graph BFS Problem Guide – Same BFS. Different Problems. Only the Changing Parts.

P1 ROTTING ORANGES (LC 994) Multi-Source BFS

CHANGED LINES

```
// 1. Push all rotten (2) into queue
// Count all fresh (1) oranges
// 2. BFS Level Order
while(!q.isEmpty() && fresh > 0) {
    int size = q.size();
    for (int i=0; i<size; i++) { /* rot fresh */ }
    minutes++;
}
return fresh == 0 ? minutes : -1;
```

AHA: Rotten oranges spread simultaneously.

Time: O(m×n) Space: O(m×n)

P2 WALLS AND GATES (LC 286) Multi-Source BFS

CHANGED LINES

```
// 1. Push all gates (0) into queue
// 2. Standard Grid BFS
while (!q.isEmpty()) {
    int[] cur = q.poll();
    for (int[] d : dirs) { /* check bounds */
        if (rooms[nr][nc] == INF) { // unvisited room
            rooms[nr][nc] = rooms[cur[0]][cur[1]] + 1;
            q.offer(new int[]{nr, nc});
        }
    }
}
```

AHA: Gates are sources. Distances expand outward.

Time: O(m×n) Space: O(m×n)

P3 SHORTEST PATH IN BINARY MATRIX Grid BFS

CHANGED LINES

```
// 1. Start at 0,0 (if valid)
// 2. Level length BFS (8 directions!)
while (!q.isEmpty()) {
    int size = q.size();
    for (int k=0; k<size; k++) {
        int[] cur = q.poll();
        if (cur[0]==n-1 && cur[1]==n-1) return steps;
        // ... add valid neighbors (8 dirs)
    }
    steps++;
}
return -1;
```

AHA: First time reaching dest = shortest path.

Time: O(n²) Space: O(n²)

P4 WORD LADDER (LC 127) Graph BFS

CHANGED LINES

```
// 1. Build graph / set of words
// 2. BFS from beginWord
while (!q.isEmpty()) {
    int size = q.size();
    for (int i=0; i<size; i++) {
        String cur = q.poll();
```

P5 NEAREST EXIT IN MAZE Grid BFS

CHANGED LINES

```
// 1. Start from entrance only
// 2. Exit = boundary cell (NOT entrance)
while (!q.isEmpty()) {
    int size = q.size();
    for (int k=0; k<size; k++) {
        // ...
```

P6 01 MATRIX (LC 542) Multi-Source BFS

CHANGED LINES

```
// 1. Push all cells with 0
// Mark 1's as unvisited
// 2. Update 1's with distance
while (!q.isEmpty()) {
    int[] cur = q.poll();
    for (int[] d : dirs) { /* bounds */
```

```
        if (cur.equals(endWord)) return steps;
        // ... loop over all 26 char changes
    }
    steps++;
}
```

AHA: BFS guarantees shortest sequence of transformations.

Time: $O(N \cdot L^2)$ Space: $O(N \cdot L)$

```
        if (isBoundary && !isEntrance) return steps;
        // ...
    }
    steps++;
}
```

AHA: BFS expands in layers. First boundary found is answer.

Time: $O(m \times n)$ Space: $O(m \times n)$

```
        if (mat[nr][nc] == 1) { // unvisited 1
            mat[nr][nc] = mat[cur[0]][cur[1]] + 1;
            q.offer(new int[]{nr, nc});
        }
    }
}
```

AHA: All zeroes start together. Distance fills matrix.

Time: $O(m \times n)$ Space: $O(m \times n)$

★ MASTER IDEA: Level Order BFS tracks steps/distance. Multi-Source BFS spreads from everywhere at once!

Next: Topological Sort Templates →

Kahn's BFS: Indegree + Queue DFS Stack: visited + recStack + Stack Min Heap: Lexicographically smallest Cycle: Cannot finish all nodes

KAHN'S ALGORITHM (BFS) ★★★★★

Idea: Remove 0 indegree nodes repeatedly. Uses: Queue + indegree[]

```
List<Integer> topoSort(int V, int[][] edges) {
    List<List<Integer>> graph = new ArrayList<>();
    for (int i=0; i<V; i++) graph.add(new ArrayList<>());
    int[] indegree = new int[V];

    // 1. Build Graph & Indegree
    for (int[] e : edges) {
        graph.get(e[0]).add(e[1]);
        indegree[e[1]]++;
    }

    // 2. Add 0 indegree nodes to queue
    Queue<Integer> q = new LinkedList<>();
    for (int i=0; i<V; i++)
        if (indegree[i] == 0) q.offer(i);

    // 3. Process
    List<Integer> order = new ArrayList<>();
    while (!q.isEmpty()) {
        int node = q.poll();
        order.add(node);
        for (int nei : graph.get(node)) {
            indegree[nei]--;
            if (indegree[nei] == 0) q.offer(nei);
        }
    }

    // 4. Cycle check
    return order.size() == V ? order : new ArrayList<>();
}
// AHA: If order.size() != V -> Cycle exists!
```

DFS + STACK (POSTORDER) ★★★★★

Idea: Push to stack *after* visiting all neighbors (postorder).

```
boolean[] visited = new boolean[V];
boolean[] recStack = new boolean[V];
Stack<Integer> stack = new Stack<>();

for (int i = 0; i < V; i++) {
    if (!visited[i])
        if (dfs(i, graph, visited, recStack, stack))
            return new ArrayList<>(); // cycle
}
// Pop from stack to get Topo Sort

boolean dfs(int node, List<List<Integer>> g,
            boolean[] vis, boolean[] rec, Stack<Integer> st) {
    vis[node] = true;
    rec[node] = true;
    for (int nei : g.get(node)) {
        if (!vis[nei]) {
            if (dfs(nei, g, vis, rec, st)) return true;
        } else if (rec[nei]) {
            return true; // cycle (back-edge)
        }
    }
    rec[node] = false;
    st.push(node);
    return false;
}
// AHA: Reversed Postorder of DFS = Topo Sort!
```

LEXICOGRAPHICALLY SMALLEST ORDER

Change: Use **PriorityQueue (Min Heap)** instead of Queue.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
// Add 0 indegree nodes to pq
for (int i=0; i<V; i++)
    if (indegree[i] == 0) pq.offer(i);

// Process
while (!pq.isEmpty()) {
    int node = pq.poll(); // gets smallest node
    // ... rest is same
}
// AHA: Min Heap ensures smallest node is picked.
```

WHEN TO USE WHICH?

Approach	Use When
Kahn's (BFS)	Graph is large, iterative preferred. Easy to count.
Kahn's + PQ	Need lexicographically smallest order.
DFS + Stack	Natural for recursion, easy cycle detection in call stack.

AHA MOMENT: Order tasks/dependencies? Think Topo Sort!

Topological Sort Problem Guide (Apply Templates)

P1 COURSE SCHEDULE

Kahn's BFS

CHANGED LINES (LC 207)

```
// 1. Graph: [a, b] -> b must be before a
// Edge: b -> a
for (int[] p : prerequisites) {
    graph.get(p[1]).add(p[0]);
    indegree[p[0]]++;
}
// 2. Return boolean if all courses taken
return count == numCourses;
```

AHA: Cycle exists? Cannot finish courses.

Time: O(V+E) Space: O(V+E)

P2 COURSE SCHEDULE II

Kahn's BFS

CHANGED LINES (LC 210)

```
// 1. Same graph construction
// 2. Store order while processing
order.add(node);
// 3. Final check
if (order.size() != numCourses)
    return new int[0]; // cycle!
return order.stream().mapToInt(i->i).toArray();
```

AHA: Kahn's gives the topological order naturally.

Time: O(V+E) Space: O(V+E)

P3 ALIEN DICTIONARY

DFS + Stack

CHANGED LINES (LC 269)

```
// 1. Build graph from adjacent words
for(int i=0; i<words.length-1; i++) {
    // find first different character
    if (w1.charAt(j) != w2.charAt(j)) {
        graph.get(w1.charAt(j)).add(w2.charAt(j));
        break;
    }
}
// 2. Handle invalid: "abc" before "ab"
```

AHA: First diff char gives edge. Topo sort gives order.

Time: O(C) Space: O(l) [26 chars]

P4 BUILD ORDER

Kahn's BFS

CHANGED LINES

```
// Given projects and dependencies
// Exact same as Course Schedule II!
// 1. Dependency [a, b] = b before a
// Edge: b -> a
// 2. Standard Kahn's
// 3. Return array if order.size() == projects
```

AHA: String nodes? Map strings to integers 0..n-1.

Time: O(V+E) Space: O(V+E)

P5 MINIMUM SEMESTERS

Kahn's (Level Order)

CHANGED LINES (LC 1136)

```
// Use BFS level-by-level logic inside Kahn's
int semesters = 0;
while (!q.isEmpty()) {
    int size = q.size();
    semesters++;
    for (int i = 0; i < size; i++) {
        // normal Kahn's poll and indegree--
    }
}
return count == n ? semesters : -1;
```

AHA: Combine Kahn's with Level Order BFS. Levels = Semesters.

Time: O(V+E) Space: O(V+E)

COMMON INTERVIEW MISTAKES

- ✗ Forgetting indegree[] initialization.
- ✗ Didn't add ALL 0 indegree nodes to queue at start.
- ✗ Used Stack instead of Queue for Kahn's.
- ✗ Didn't check order.size() == V for cycles!
- ✗ Confusing 0 indegree concept (these are roots).
- ✗ Not handling disconnected DAG components.

★ MASTER IDEA: Topological Sort = Valid Linear Order of Tasks. If impossible -> CYCLE EXISTS.

Next: Union Find & Dijkstra Templates ->

UNION FIND TEMPLATE (OPTIMIZED DSU) ★★★★★

Idea: Manage disjoint sets. Fast cycle detection (undirected) and dynamic connectivity.

```
class UnionFind {
    int[] parent;
    int[] rank;
    int count; // num components

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        count = n;
        for (int i=0; i<n; i++)
            parent[i] = i;

        // 1. Find with Path Compression
        public int find(int x) {
            if (parent[x] != x)
                parent[x] = find(parent[x]);
            return parent[x];
        }

        // 2. Union by Rank
        public boolean union(int a, int b) {
            int rootA = find(a);
            int rootB = find(b);
            if (rootA == rootB) return false; // Cycle!

            if (rank[rootA] < rank[rootB]) {
                parent[rootA] = rootB;
            } else if (rank[rootA] > rank[rootB]) {
                parent[rootB] = rootA;
            } else {
                parent[rootB] = rootA;
                rank[rootA]++;
            }
            count--;
            return true;
        }
    } // Complexity: O(a(N)) = O(1) Amortized
}
```

UNION FIND PROBLEMS

1. Redundant Connection (LC 684)
For each edge (u,v): If !union(u, v) return edge. It's the cycle!

2. Graph Valid Tree (LC 261)
Must have: edges == n-1
And: No cycles (all unions return true) & count == 1.

3. Number of Provinces (LC 547)
For each 1 in matrix: union(i, j). Return count.

4. Making A Large Island (LC 827)
Use size[] instead of rank. Map grid ID → Root. Flip 0 to 1 and sum unique neighbor root sizes + 1.

DIJKSTRA'S SHORTEST PATH TEMPLATE (ADJACENCY LIST) ★★★★★

Idea: Find shortest path from a source to all other nodes (Non-negative weights!).

```
class Pair { int node, dist; ... }

public int[] dijkstra(List<List<Pair>> graph,
                    int n, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;

    PriorityQueue<Pair> pq = new PriorityQueue<>()
        (a, b) → a.dist - b.dist // Min Heap by dist
    );
    pq.offer(new Pair(src, 0));

    boolean[] visited = new boolean[n];

    while (!pq.isEmpty()) {
        Pair cur = pq.poll();
        int u = cur.node, d = cur.dist;

        // Important: skip already finalized nodes
        if (visited[u]) continue;
        visited[u] = true;

        for (Pair nei : graph.get(u)) {
            int v = nei.node, weight = nei.dist;
            // Relaxation Step
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.offer(new Pair(v, dist[v]));
            }
        }
    }
    return dist;
} // Complexity: O((V + E) log V)
```

DIJKSTRA COMMON ERRORS

- ✗ Using with negative edges. (Dijkstra assumes distances only grow).
- ✗ Forgetting visited[] check. (Same node might be pushed multiple times with different distances. Only expand it once).
- ✗ Integer Overflow. (Use long[] dist if weights can sum up past 2^31-1).

WHEN TO USE DIJKSTRA?

- ✓ Edge weights are non-negative.
- ✓ Need shortest path from ONE source to all nodes.
- ✓ Can be directed or undirected.
- ✗ If unweighted → Use BFS.
- ✗ If negative weights → Use Bellman-Ford.

